SOFTWARE ENGINEERING PROJECT

UrbanHive

Housing Society Management System

MIT World Peace University, Pune · 2025–2026 · SE & Modelling

Sr. No.	Student Name	PRN No.	Roll No.
1	Kunal Tailor	1032233258	38
2	Rishav Singh	1032233012	34
3	Prakash	1032233277	40
4	Anuj Kudu	1032233203	36

Project Guide

Dr. Yogesh Kulkarni

PROBLEM STATEMENT

Why UrbanHive Exists

Manual Registers

Complaint books, payment ledgers, and attendance sheets are error-prone and impossible to audit at scale.

Fragmented Comms

WhatsApp groups and verbal notices leave residents without a structured, role-filtered announcement channel.

No Accountability

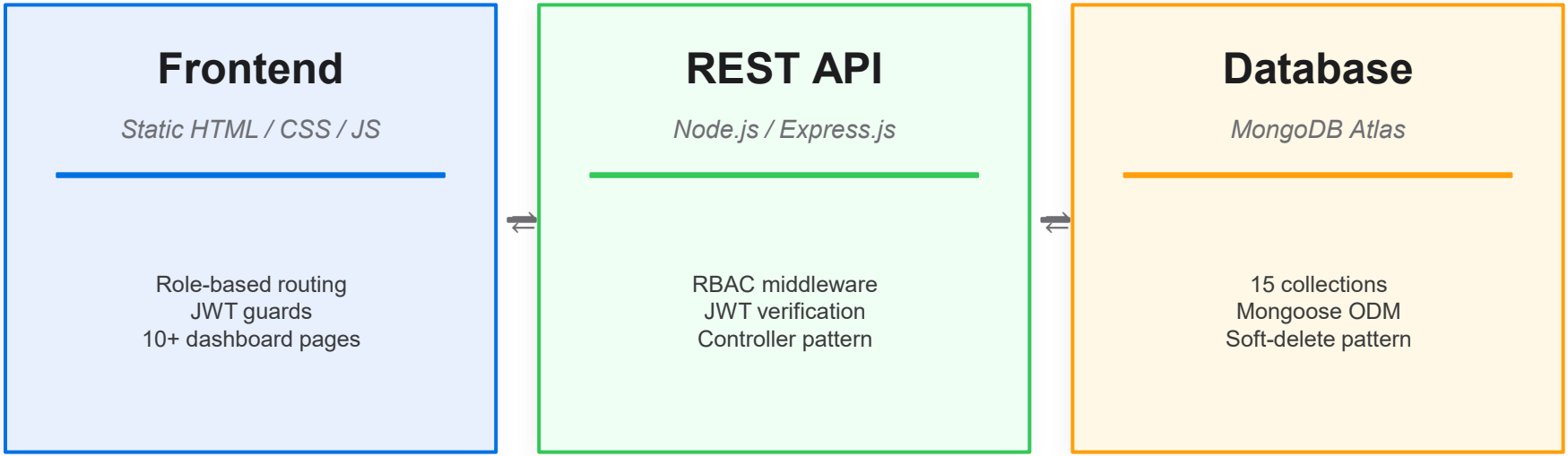
No complaint ownership, no status trail, no payment transparency — residents are left in the dark.

Technical Gaps in Existing Approaches

- No centralized authentication — shared logins undermine accountability
- Role mixing — general apps like Google Sheets don't enforce access boundaries
- No workflow state machine — complaints don't move through defined lifecycle states
- No API contract — frontend & backend can't be developed or tested independently

SYSTEM ARCHITECTURE

Three-Tier, Role-Aware Architecture



🔐 RBAC enforced at two levels — Frontend hides UI elements based on JWT role · Backend independently rejects unauthorized requests on every endpoint

MULTI-ROLE SYSTEM

Three Roles. One Platform.

Resident

- Raise & track complaints
- Submit maintenance payments
- Request visitor approvals
- View notices & events
- Book society amenities

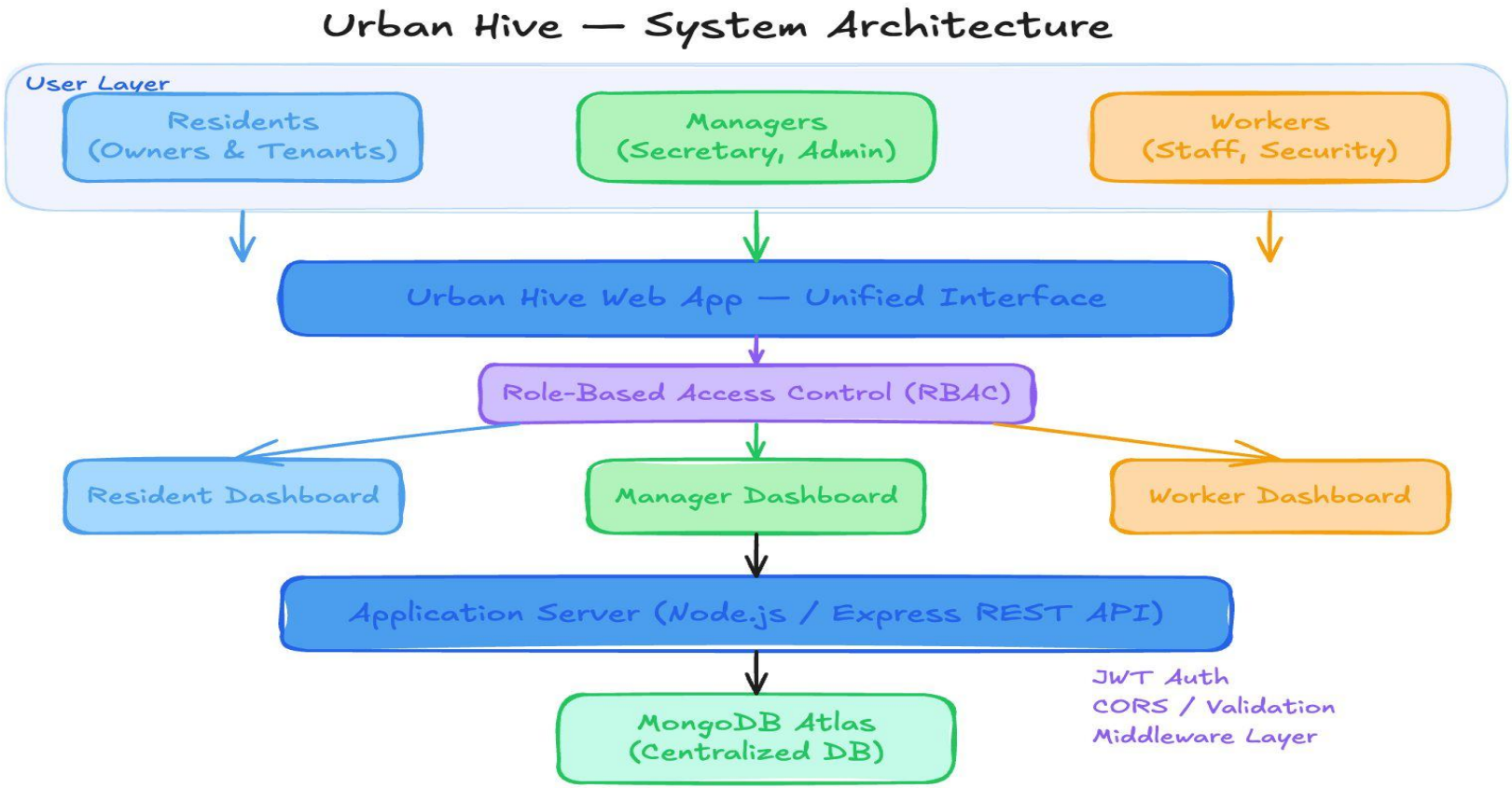
Manager

- Assign complaints to workers
- Verify payment submissions
- Approve visitor requests
- Publish notices
- Monitor worker attendance

Worker

- View assigned tasks
- Mark attendance in/out
- Report task completion
- Receive staff-tagged notices
- Update availability

System Architecture



PROJECT MODULES

Five Core Modules

01

Auth & RBAC

Phase 2 ✓

JWT login, bcrypt hashing, role guards on every protected endpoint. Server-side RBAC independent of frontend.

02

Core Workflows

Phase 3 →

Complaints lifecycle (Open→Resolved), payment verification, visitor management, and notice board.

03

Secondary Modules

Phase 4

Event management, amenity booking (with availability checks), profile management, and notification fanout.

04

UI Integration

Phase 5

Static frontend connected to live API. Centralized API client with JWT injection, error handling, and loading states.

05

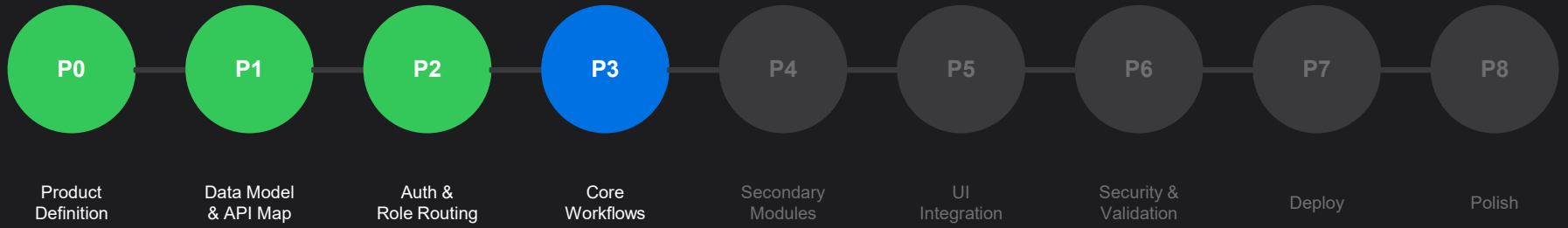
Security & Deploy

Phase 6–7

Input validation (express-validator), CORS hardening, rate limiting, Render deployment with MongoDB Atlas.

SOFTWARE PROCESS MODEL

Incremental Delivery Model



Why Incremental over Waterfall?

Early Working Software	✓ After each phase	✗ Only at the end
Handles Evolving Requirements	✓ Later phases absorb changes	✗ Scope freeze required
User Feedback	✓ Per increment	✗ Only post-delivery
Risk Management	✓ High-risk first	✗ Risks surface late

TECHNOLOGY STACK

Tools & Technologies

Frontend

— HTML / CSS / JavaScript (Vanilla)

— Role-aware page routing with JS guards

— JWT decode for conditional rendering

Backend

— Node.js 20+ · Express.js

— JWT (jsonwebtoken) · bcrypt

— express-validator · dotenv · nodemon

Database

— MongoDB Atlas (Cloud)

— Mongoose ODM — 15 collections

— Connection pooling · IP allowlisting

Auth & Security

— JWT stateless token auth

— RBAC middleware (server-side)

— CORS config · Rate limiting

DevOps

— Render cloud hosting

— Environment variables (never in source)

— GET /health endpoint · GitHub

DATABASE DESIGN

15 MongoDB Collections — 5 Logical Groups

Core / Auth	Resident	Operations	Worker	Comms
Users	Residents	Complaints	Workers	Notifications
Roles	Flats	Payments	Tasks	Managers
Permissions		Visitor_Logs	Attendance	
Role_Permissions		Notices		

Key Design Decisions

- Single Users table for all roles — role-specific data in separate linked collections
- Complaint status as ENUM (Open → Assigned → InProgress → Resolved) for state machine integrity
- Soft-delete pattern — records are never hard-deleted, preserving a complete audit trail

RESULTS

What We've Built

3	15	9	10+
User Roles RBAC Enforced	MongoDB Collections	Development Phases	Static Frontend Pages

- ✓ Phase-0: Product definition, role matrix, feature lock
- ✓ Phase-1: Full MongoDB schema + API endpoint specification
- ✓ Phase-2: JWT auth, bcrypt, RBAC middleware, 3 demo accounts
- → Phase-3: Core workflows — complaints, payments, visitors, notices
- ✓ GET /health — server + DB connectivity confirmed
- ✓ 10+ frontend pages with role-based conditional rendering

CHALLENGES

Challenges & How We Solved Them

RBAC on Static Frontend

⚠ Users could modify localStorage to bypass JS guards.

✓ Server-side RBAC on every endpoint makes frontend bypass irrelevant.

JWT Expiry UX

⚠ Silent token expiry caused confusing mid-session 401 errors.

✓ Token expiry checked before each API call with auto-redirect to login.

CORS Dev vs Production

⚠ localhost and Render URLs require different CORS origins.

✓ CORS origin read from environment variables with localhost fallback.

Static Frontend Routing

⚠ Direct URL navigation bypasses login guards on HTML pages.

✓ Guard check at top of every protected page script — redirects if no token.

Phase-3 Scope Size

⚠ Four major workflows in one increment was too large.

✓ Subdivided into Sprint 3A (complaints + payments) and 3B (visitors + notices).

Mongoose Schema Sync

⚠ Schema changes during Phase-1 conflicted with Phase-2 test data.

✓ Schema version policy: all Phase-1 schema changes finalized before Phase-2 begins.

ROADMAP

Future Scope

Real Payment Gateway

Razorpay / Stripe for online maintenance collection with auto-reconciliation.

AI Complaint Triage

LLM classifier to auto-tag Emergency complaints and suggest assignment.

Mobile App (React Native)

Native iOS/Android wrapping the same REST API with FCM push notifications.

QR Visitor Entry

QR code generation for pre-approved guests, scanned at gate by security.

Online Voting

Digital elections for committee roles with tamper-evident vote records.

Multi-Society SaaS

Multiple societies as isolated tenants under one platform.

Predictive Maintenance

AMC schedule tracking for lifts, generators — alert managers before due dates.

IoT Sensor Integration

Water tank levels and generator health on Manager Dashboard in real time.

TEAM

Project Team

Kunal Tailor

Lead Engineer & System Architect

PRN: 1032233258 Roll: 38

Overall architecture, Phase-2 JWT/RBAC, Phase-3 complaint & payment APIs, MongoDB schema, documentation.

Rishav Singh

Backend Developer

PRN: 1032233012 Roll: 34

Phase-1 API map, Mongoose models, Phase-3 visitor & notice APIs, Phase-6 validation, Render deployment.

Prakash

Frontend Developer

PRN: 1032233277 Roll: 40

All role dashboards, login/register UI, complaint & payment components, responsive layout, routing guards.

Anuj Kudu

Testing & Documentation

PRN: 1032233203 Roll: 36

Phase-5 integration testing, bug log, regression tracking, user docs, system design, final presentation.

Guide: Yogesh Kulkarni · MIT World Peace University, Pune

CONCLUSION

UrbanHive

Housing Society Management, Digitised.

A clean three-tier, role-aware architecture proving that open-source technologies — Node.js, Express, MongoDB, and vanilla JavaScript — are sufficient to build a production-grade, privacy-conscious society management platform.